

Деревья решений: сложность, важность признаков и эксперименты

ИТМО · Машинное обучение · Практика · 90 минут

На реальных измерениях клеточных ядер построим классификатор двух классов, посмотрим на его правила и проверим, почему хорошее качество на обучении ещё не гарантирует хорошее качество на новых объектах.

После занятия вы сможете выбрать сложность дерева по validation, объяснить разницу между MDI, permutation importance и SHAP и воспроизвести запуск по конфигу.

Блок	Минуты
Данные, разбиение и чтение дерева	15
Границы решений, глубина и переобучение	20
Признаки, MDI и permutation importance	20
Два равных дерева и разные важности	10
ELI5 и локальное объяснение SHAP	12
Конфиги, MLflow и финальная проверка	13

Перед запуском графика сначала ответьте на вопрос над ячейкой. В заданиях меняйте указанные параметры и записывайте выводы; готовые функции для рисования и сохранения запусков можно использовать без переписывания.

Подготовка

Ноутбук открывается в Jupyter или Colab. В архиве есть данные и конфиги. Если открыт только этот `.ipynb`, данные загрузятся из встроенного набора `scikit-learn`.

До занятия установите зависимости. На первом запуске при необходимости раскомментируйте следующую строку и перезапустите ядро после установки.

Код 1

```
# %pip install "numpy>=1.26,<3" "pandas>=2.2,<3" "matplotlib>=3.8,<4" "scikit-learn>=1.6,<1.10" "shap>=0.48,<0.51" "eli5==0.16.0"
"mlflow>=3.0,<4"
```

Код 2

```
from pathlib import Path
import hashlib
import json
import os
import platform
import importlib.metadata

os.environ["MLFLOW_DISABLE_AGENT_HINT"] = "1"
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display
from sklearn.datasets import load_breast_cancer
from sklearn.dummy import DummyClassifier
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.inspection import permutation_importance
from sklearn.metrics import (balanced_accuracy_score, accuracy_score,
                             recall_score, ConfusionMatrixDisplay)

plt.rcParams.update({"figure.dpi": 115, "font.size": 11, "axes.spines.top": False,
                    "axes.spines.right": False})
ROOT = next((p for p in [Path.cwd(), Path.cwd().parent, Path.cwd()/"tree_practice"]
             if (p/"data"/"wdbc.csv").exists()), Path.cwd())
OUT = ROOT / "runs"
FIGURES = ROOT / "figures"
OUT.mkdir(exist_ok=True)
FIGURES.mkdir(exist_ok=True)
SEED = 42
CLASS_NAMES = ["Доброкачественный", "Злокачественный"]

def save_plot(name):
    plt.gcf().savefig(FIGURES / f"{name}.png", dpi=150, bbox_inches="tight")
    plt.show()
```

1. Реальные данные и разбиение

Wisconsin Diagnostic Breast Cancer (WDBC): 569 образцов, 30 числовых признаков. Изображения получены при тонкоигольной аспирации; признаки описывают клеточные ядра. Для десяти характеристик записаны среднее (mean), стандартная ошибка (error) и среднее трёх крупнейших значений (worst).

Название	Смысл
radius	Среднее расстояние от центра до границы ядра
texture	Стандартное отклонение яркости
perimeter / area	Периметр / площадь
smoothness	Локальные изменения радиуса
compactness	Периметр ² / площадь – 1
concavity / concave points	Выраженность / число вогнутых участков
symmetry / fractal dimension	Симметрия / сложность границы

Цель: target = 1 — злокачественный класс, target = 0 — доброкачественный. **В scikit-learn исходная кодировка обратная; ниже мы явно её меняем.** Вероятности дерева описывают модель на этом наборе, а не клинический риск.

Источник: [UCI](#), DOI [10.24432/C5DW2B](#). Авторы: Wolberg, Mangasarian, Street & Street (1993). Лицензия CC BY 4.0.

Код 3

```
data_path = ROOT / "data" / "wdbc.csv"
if data_path.exists():
    data = pd.read_csv(data_path)
else:
    original = load_breast_cancer(as_frame=True)
    data = original.data.copy()
    data["target"] = (original.target == 0).astype(int)

X = data.drop(columns="target")
y = data["target"].astype(int)
display(data.head(3))
print(f"Объекты: {len(X)}; признаки: {X.shape[1]}; пропуски: {X.isna().sum().sum()}")
display(y.value_counts().sort_index().rename(index=dict(enumerate(CLASS_NAMES))).to_frame("Количество"))

train_idx, test_idx = train_test_split(X.index, test_size=0.20, stratify=y, random_state=SEED)
train_idx, val_idx = train_test_split(train_idx, test_size=0.25,
```

```

stratify=y.loc[train_idx], random_state=SEED)
X_train, X_val, X_test = X.loc[train_idx], X.loc[val_idx], X.loc[test_idx]
y_train, y_val, y_test = y.loc[train_idx], y.loc[val_idx], y.loc[test_idx]
display(pd.DataFrame({"Объектов": [len(train_idx), len(val_idx), len(test_idx)],
                             "Доля класса 1": [y_train.mean(), y_val.mean(), y_test.mean()]},
                    index=["train", "validation", "test"]).round(3))

```

Объекты: 569; признаки: 30; пропуски: 0

	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis	mean ratio dis
0	17 .99	10 .38	12 2.8	10 01.0	0.11 840	0.277 60	0.301 01	0.147 10	0.2419	0.078 71	1.09 50	0.09 53	8.58 90	1.03 40	0.0063 99	0.049 5	0.058 37	0.0300 3	0.0019 3	25 38	17 33	18 4.6	2 01 9.0	0.162 2	0.665 6	0.071 19	0.026 54	0.046 01	0.118 90	1
1	20 57	17 77	13 2.9	13 26.0	0.08 474	0.078 64	0.08 69	0.070 17	0.1812	0.056 67	0.05 43 5	0.07 33 9	3.39 8	7.4 08 25	0.00 52	0.013 08	0.01 134 80	0.01 389	0.0353 2	24 99	23 41	15 8	1 5 6.0	0.123 8	0.186 6	0.024 16	0.018 60	0.027 50	0.089 02	1
2	19 69	21 25	13 0.0	12 03.0	0.10 960	0.159 90	0.119 74	0.2090	0.059 99	0.07 45 6	0.07 86 9	4.58 5	9.4 03 50	0.00 61 50	0.040 06	0.03 058 3	0.02 250	0.02 250	0.0457 1	23 57	25 53	15 2.5	1 0 9.0	0.144 4	0.424 5	0.045 04	0.024 30	0.036 13	0.087 58	1

	Количество
target	
Доброкачественный	357
Злокачественный	212

	Объектов	Доля класса 1
train	341	0.372
validation	114	0.377
test	114	0.368

На train обучаемся, по validation сравниваем настройки и исследуем модель. **Test откроем один раз, после выбора модели в конце занятия.**

Основная метрика — **balanced accuracy**: среднее recall двух классов. Значение 0,5 соответствует постоянному прогнозу одного класса, если оба класса присутствуют. Дополнительно считаем accuracy и recall класса 1. Во всех сравнениях используется одно и то же разбиение.

Код 4

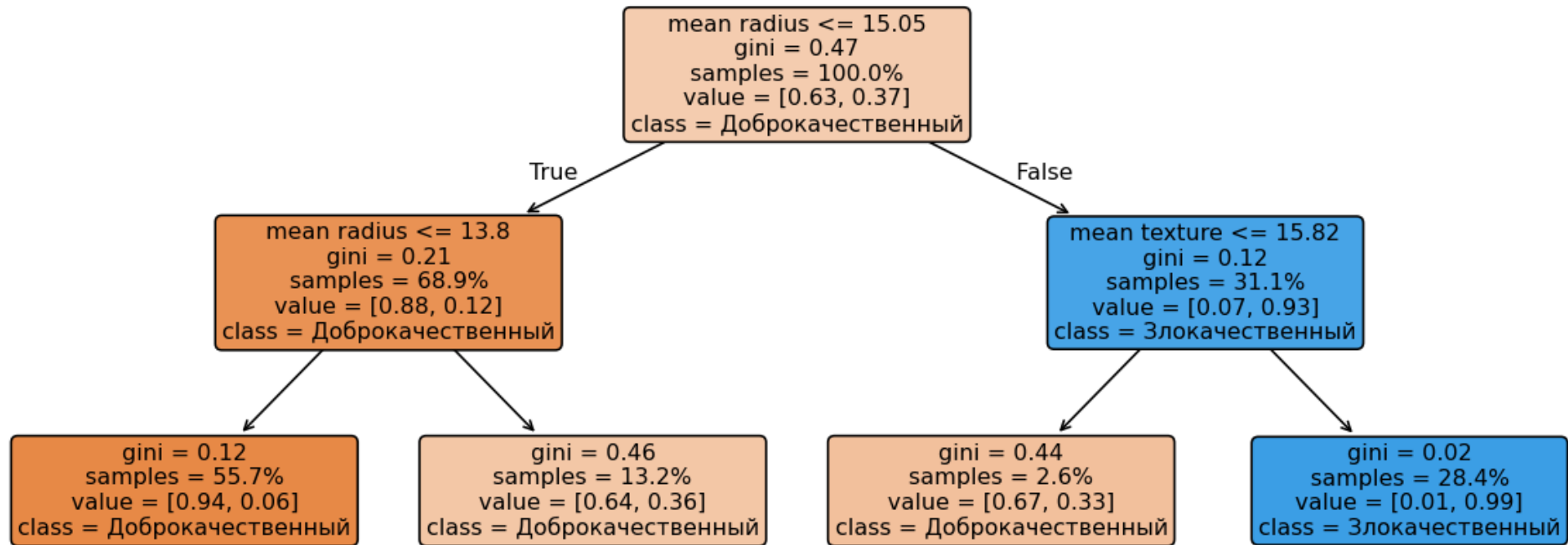
```
def metrics(model, features, target):
    pred = model.predict(features)
    return {"balanced_accuracy": balanced_accuracy_score(target, pred),
            "accuracy": accuracy_score(target, pred),
            "recall_class_1": recall_score(target, pred, zero_division=0)}

baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
display(pd.DataFrame([metrics(baseline, X_val, y_val)], index=["Постоянный прогноз"]).round(3))

two_features = ["mean radius", "mean texture"]
small_tree = DecisionTreeClassifier(max_depth=2, random_state=SEED)
small_tree.fit(X_train[two_features], y_train)
fig, ax = plt.subplots(figsize=(13, 5))
plot_tree(small_tree, feature_names=two_features, class_names=CLASS_NAMES,
          filled=True, rounded=True, proportion=True, precision=2, ax=ax, fontsize=10)
ax.set_title("Дерево по двум признакам: один путь – одно правило")
save_plot("01_tree")
```

	balanced_accuracy	accuracy	recall_class_1
Постоянный прогноз	0.5	0.623	0.0

Дерево по двум признакам: один путь — одно правило



Задание 1. Прочитайте дерево. Возьмите первый объект validation. До выполнения следующей ячейки найдите его путь от корня до листа на рисунке. Назовите прогноз класса. Все ли 30 признаков использует эта модель?

Код 5

```
sample = X_val[two_features].iloc[[0]]
display(sample)
```

	mean radius	mean texture
488	11.68	16.17

Разбор для преподавателя: Модель использует только mean radius и mean texture. Путь определяется последовательными сравнениями с порогами, а не близостью к центру класса. Точный путь и прогноз приведены в следующей ячейке.

Код 6

```
nodes = small_tree.decision_path(sample).indices
path = []
for node in nodes:
    feature_id = small_tree.tree_.feature[node]
    if feature_id >= 0:
        feature = two_features[feature_id]
        value = float(sample.iloc[0, feature_id])
        threshold = small_tree.tree_.threshold[node]
        path.append({"Признак": feature, "Значение": value, "Порог": threshold,
                    "Ветка": "влево (<=" if value <= threshold else "вправо (>)"})
display(pd.DataFrame(path).round(3))
print("Прогноз:", CLASS_NAMES[int(small_tree.predict(sample)[0])])
print("Доля класса 1 в листе:", round(small_tree.predict_proba(sample)[0, 1], 3))
```

Прогноз: Доброкачественный
Доля класса 1 в листе: 0.063

	Признак	Значение	Порог	Ветка
0	mean radius	11.68	15.045	влево ($\leq$)
1	mean radius	11.68	13.795	влево ($\leq$)

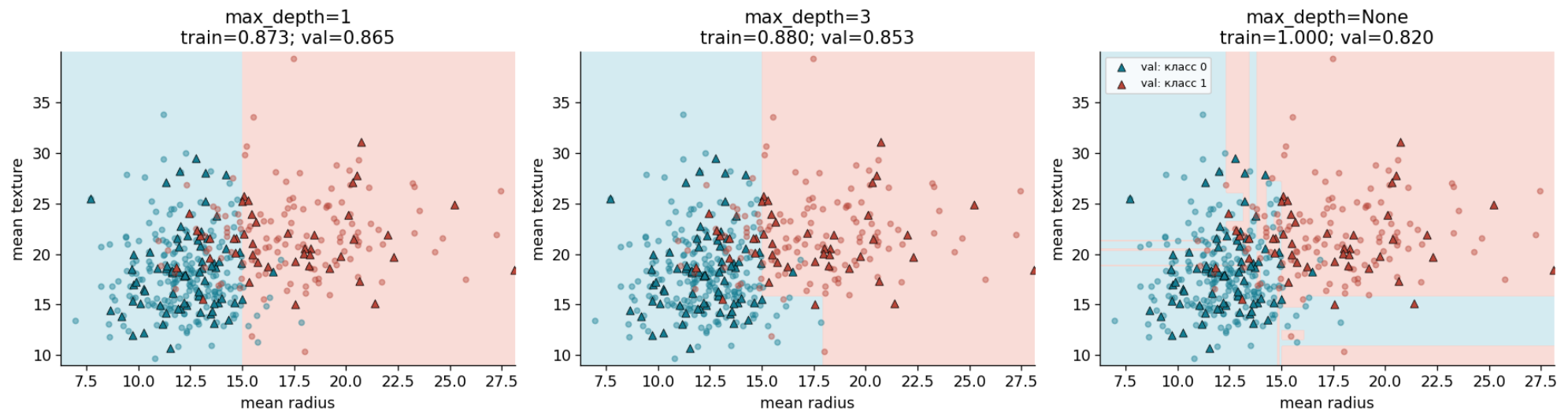
2. Границы решений и переобучение

Начнём с **двух признаков**, чтобы видеть границу напрямую. Эти модели действительно обучены на двух колонках; график не является двумерной проекцией модели с 30 признаками.

Перед запуском: какое дерево даст самое высокое качество на train? Будет ли оно лучшим на validation? Какую форму имеют границы?

```
def draw_boundary(model, ax, title):
    a, b = two_features
    xlim = (X_train[a].min()-0.7, X_train[a].max()+0.7)
    ylim = (X_train[b].min()-0.7, X_train[b].max()+0.7)
    xx, yy = np.meshgrid(np.linspace(*xlim, 240), np.linspace(*ylim, 240))
    grid = pd.DataFrame({a: xx.ravel(), b: yy.ravel()})
    zz = model.predict(grid).reshape(xx.shape)
    ax.contourf(xx, yy, zz, levels=[-0.5, 0.5, 1.5], colors=["#cae7ee", "#f8d4cd"], alpha=.8)
    for cls, color in [(0, "#147d92"), (1, "#b94639")]:
        tr, va = y_train == cls, y_val == cls
        ax.scatter(X_train.loc[tr, a], X_train.loc[tr, b], c=color, s=15, alpha=.42)
        ax.scatter(X_val.loc[va, a], X_val.loc[va, b], c=color, marker="^", s=34,
                  edgecolors="black", linewidths=.45, label=f"val: класс {cls}")
    ax.set(xlabel=a, ylabel=b, title=title, xlim=xlim, ylim=ylim)

fig, axes = plt.subplots(1, 3, figsize=(16, 4.4))
for depth, ax in zip([1, 3, None], axes):
    m = DecisionTreeClassifier(max_depth=depth, random_state=SEED).fit(X_train[two_features], y_train)
    tr = metrics(m, X_train[two_features], y_train)["balanced_accuracy"]
    va = metrics(m, X_val[two_features], y_val)["balanced_accuracy"]
    draw_boundary(m, ax, f"max_depth={depth}\ntrain={tr:.3f}; val={va:.3f}")
axes[-1].legend(fontsize=8, loc="upper left")
plt.tight_layout()
save_plot("02_boundaries")
```

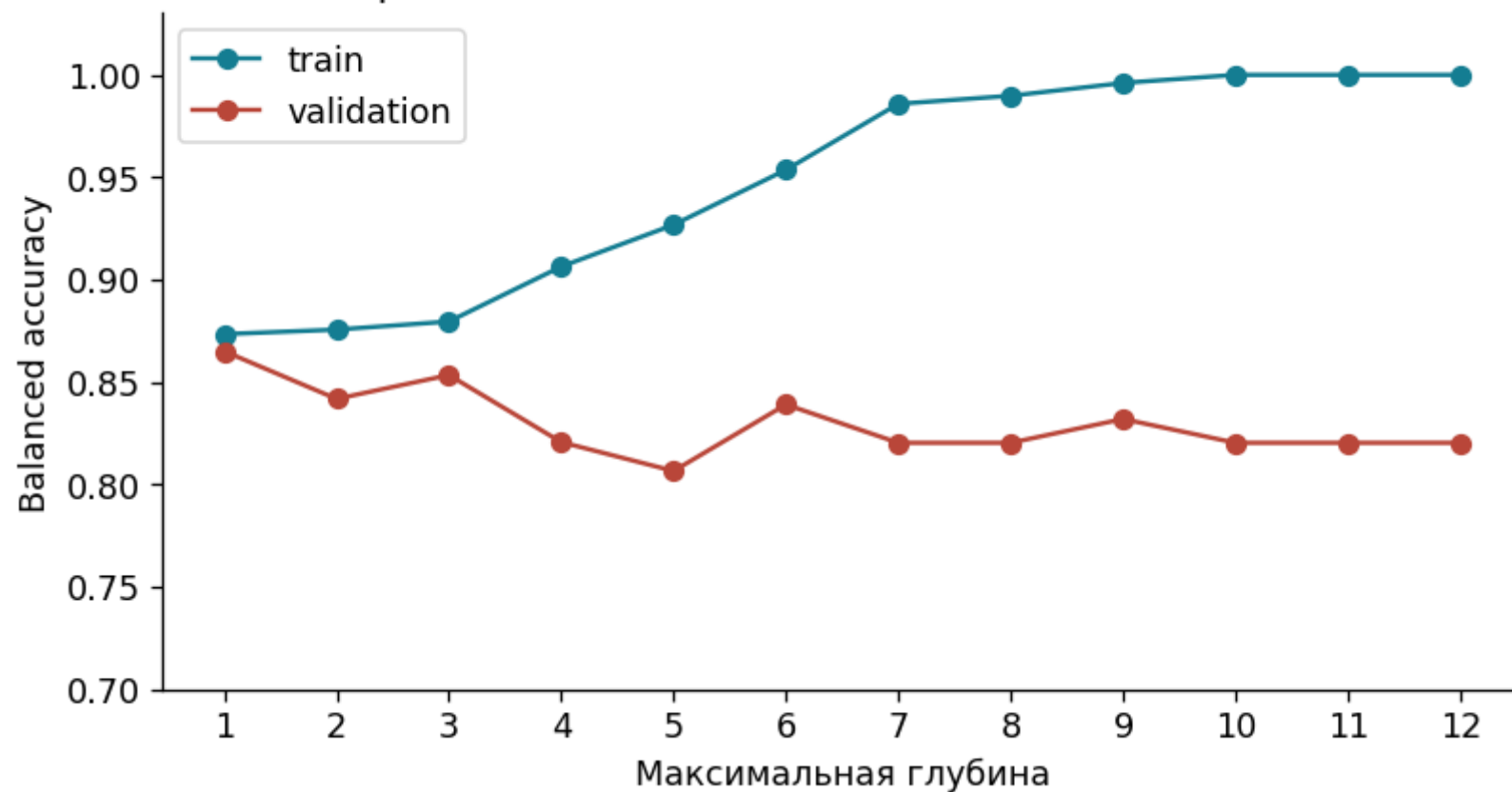


Дальше перебираем глубины, не меняя данные. На каждом запуске записываем train/validation balanced accuracy, фактическую глубину и число листьев.

Код 8

```
depth_rows = []
for depth in range(1, 13):
    m = DecisionTreeClassifier(max_depth=depth, random_state=SEED).fit(X_train[two_features], y_train)
    depth_rows.append({"max_depth": depth, "train": metrics(m, X_train[two_features], y_train)["balanced_accuracy"],
                      "validation": metrics(m, X_val[two_features], y_val)["balanced_accuracy"],
                      "leaves": m.get_n_leaves(), "actual_depth": m.get_depth()})
depth_table = pd.DataFrame(depth_rows)
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(depth_table.max_depth, depth_table.train, "o-", label="train", color="#147d92")
ax.plot(depth_table.max_depth, depth_table.validation, "o-", label="validation", color="#b94639")
ax.set(xlabel="Максимальная глубина", ylabel="Balanced accuracy", ylim=(.7, 1.03),
       xticks=range(1, 13), title="Сложность растёт; качество на новых объектах – не обязательно")
ax.legend()
save_plot("03_depth_curve")
display(depth_table.round(3))
```

Сложность растёт; качество на новых объектах — не обязательно



	max_depth	train	validation	leaves	actual_depth
0	1	0.873	0.865	2	1
1	2	0.876	0.842	4	2
2	3	0.880	0.853	8	3
3	4	0.906	0.821	14	4
4	5	0.927	0.807	21	5
5	6	0.954	0.839	30	6
6	7	0.986	0.820	40	7

	max_depth	train	validation	leaves	actual_depth
7	8	0.990	0.820	43	8
8	9	0.996	0.832	47	9
9	10	1.000	0.820	48	10
10	11	1.000	0.820	48	10
11	12	1.000	0.820	48	10

Задание 2. Ограничьте переобучение. Меняйте YOUR_DEPTH и YOUR_MIN_LEAF. Сравните не менее трёх настроек с глубоким деревом. Не меняйте разбиение. Запишите параметры, train/validation quality и вывод: какую модель вы предпочли бы и почему?

min_samples_leaf задаёт минимальное число **обучающих объектов** в листе. Это ещё один способ ограничить сложность, помимо глубины.

Код 9

```
YOUR_DEPTH = 3
YOUR_MIN_LEAF = 5

settings = [(None, 1), (YOUR_DEPTH, YOUR_MIN_LEAF), (None, 10)]
leaf_rows = []
for depth, leaf in settings:
    m = DecisionTreeClassifier(max_depth=depth, min_samples_leaf=leaf, random_state=SEED)
    m.fit(X_train[two_features], y_train)
    leaf_rows.append({"max_depth": str(depth), "min_samples_leaf": leaf, "leaves": m.get_n_leaves(),
                     "train": metrics(m, X_train[two_features], y_train)["balanced_accuracy"],
                     "validation": metrics(m, X_val[two_features], y_val)["balanced_accuracy"]})
display(pd.DataFrame(leaf_rows).round(3))
```

	max_depth	min_samples_leaf	leaves	train	validation
0	None	1	48	1.000	0.820
1	3	5	7	0.876	0.842
2	None	10	16	0.896	0.821

Разбор для преподавателя: На фиксированном разбиении полное дерево по двум признакам запоминает train (balanced accuracy 1.000), но validation ≈ 0.820 . Глубина 1 даёт ≈ 0.865 на validation. min_samples_leaf уменьшает число листьев, но улучшение validation не гарантировано для каждой настройки. Выбираем по validation, не по train; на маленькой validation небольшие различия могут быть шумом. Это не доказательство универсальной оптимальности глубины 1.

3. Меняем признаки: две колонки и все 30

Сохраняем разбиение и одинаковые настройки дерева. Добавление признаков может дать полезный сигнал, но качество надо проверить. Дерево не требует стандартизации только из-за разных единиц измерения признаков.

Сравним заранее заданные три конфигурации по двум наборам признаков. Для исследования важности возьмём all_shallow: компактное дерево по всем признакам, которое использует несколько колонок. Это позволяет разобрать несколько вкладов. Итоговую модель выберем по validation в конце занятия. Это небольшой учебный перебор, не полный поиск.

Код 10

```
candidates = []
for group, cols in [("two", two_features), ("all", list(X.columns))]:
    for name, depth, leaf in [("stump", 1, 1), ("shallow", 3, 5), ("deep", None, 1)]:
        m = DecisionTreeClassifier(max_depth=depth, min_samples_leaf=leaf, random_state=SEED)
        m.fit(X_train[cols], y_train)
        config = {"run_name": f"{group}_{name}", "features": group, "max_depth": depth,
                  "min_samples_leaf": leaf, "ccp_alpha": 0.0, "split_seed": SEED, "model_seed": SEED}
        candidates.append({"name": config["run_name"], "cols": cols, "model": m, "config": config,
                           "train": metrics(m, X_train[cols], y_train)["balanced_accuracy"],
                           "validation": metrics(m, X_val[cols], y_val)["balanced_accuracy"],
                           "leaves": m.get_n_leaves()})
comparison = pd.DataFrame([{"k": c[k] for k in ["name", "train", "validation", "leaves"]} for c in candidates])
display(comparison.sort_values(["validation", "leaves"], ascending=[False, True]).round(3))
chosen = next(c for c in candidates if c["name"] == "all_shallow")
model, columns = chosen["model"], chosen["cols"]
print("Модель для исследования важности:", chosen["name"])
```

Модель для исследования важности: all_shallow

	name	train	validation	leaves
3	all_stump	0.918	0.916	2

	name	train	validation	leaves
4	all_shallow	0.965	0.914	7
5	all_deep	1.000	0.904	17
0	two_stump	0.873	0.865	2
1	two_shallow	0.876	0.842	7
2	two_deep	1.000	0.820	48

MDI и permutation importance

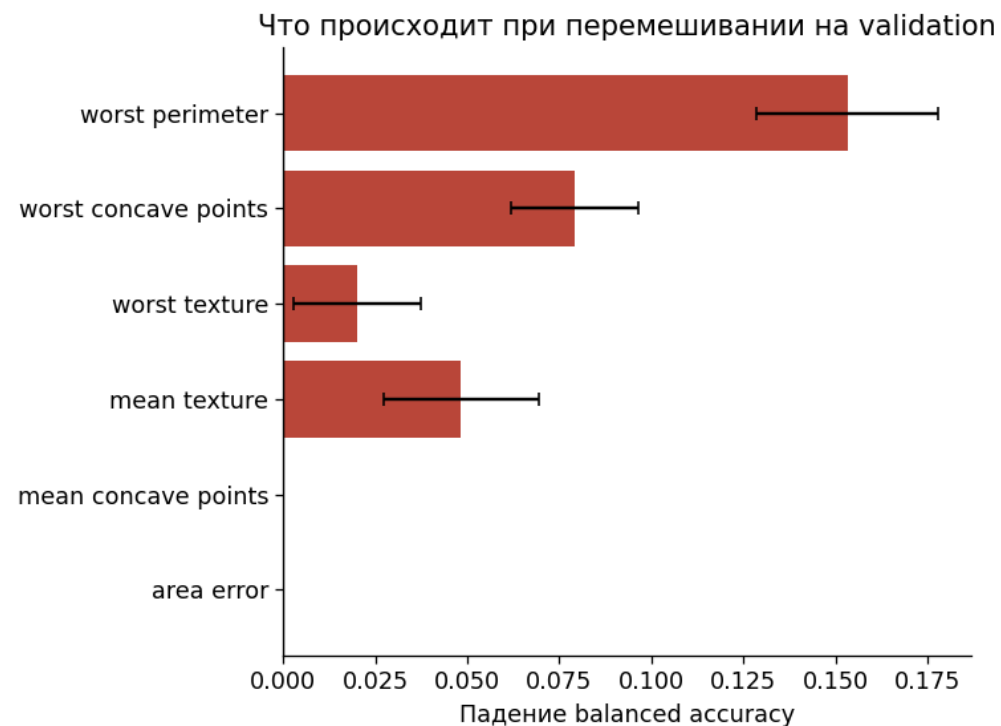
MDI суммирует взвешенные уменьшения impurity в обученном дереве и нормирует их сумму до 1. Это характеристика разбиений на train.

Permutation importance (PI): фиксируем модель, перемешиваем одну колонку на validation и считаем падение выбранной метрики. Модель не переобучаем. Повторяем перемешивание 20 раз.

Ни MDI, ни PI не являются доказательством причинного влияния признака. Единицы отличаются: доля уменьшения impurity против изменения balanced accuracy.

Код 11

```
pi = permutation_importance(model, X_val[columns], y_val,
                           scoring="balanced_accuracy", n_repeats=20, random_state=SEED)
importance = pd.DataFrame({"MDI": model.feature_importances_, "PI_mean": pi.importances_mean,
                          "PI_shuffle_sd": pi.importances_std}, index=columns)
shown = importance[importance.MDI > 0].sort_values("MDI")
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
axes[0].barh(shown.index, shown.MDI, color="#147d92")
axes[0].set(xlabel="MDI (сумма по всем признакам = 1)", title="Какие признаки использовало дерево")
axes[1].barh(shown.index, shown.PI_mean, xerr=shown.PI_shuffle_sd, color="#b94639", capsize=3)
axes[1].axvline(0, color="black", linewidth=.8)
axes[1].set(xlabel="Падение balanced accuracy", title="Что происходит при перемешивании на validation")
plt.tight_layout()
save_plot("04_importance")
display(importance.sort_values("MDI", ascending=False).head(12).round(4))
```



	MDI	PI_mean	PI_shuffle_sd
worst perimeter	0.7949	0.1532	0.0246
worst concave points	0.0801	0.0791	0.0171
worst texture	0.0588	0.0201	0.0173
mean texture	0.0431	0.0483	0.0210
mean concave points	0.0127	0.0000	0.0000
area error	0.0103	0.0000	0.0000
mean compactness	0.0000	0.0000	0.0000
mean smoothness	0.0000	0.0000	0.0000
mean symmetry	0.0000	0.0000	0.0000
mean concavity	0.0000	0.0000	0.0000

	MDI	PI_mean	PI_shuffle_sd
mean fractal dimension	0.0000	0.0000	0.0000
mean area	0.0000	0.0000	0.0000

На графиках показаны признаки, использованные деревом; таблица включает и нулевые важности. На графике справа усы показывают **стандартное отклонение по перемешиваниям**. Это не доверительный интервал по новым выборкам и не оценка всей неопределённости модели.

Задание 3. Интерпретируйте важность. Назовите первые три признака по каждому методу. Совпадает ли порядок? Что означает PI около нуля или отрицательное значение? Можно ли удалить все признаки с низким PI без дополнительной проверки?

Разбор для преподавателя: Порядки могут различаться, потому что методы отвечают на разные вопросы и используют разные данные. PI около нуля означает, что конкретная модель на этой validation и для этой метрики мало теряет при перемешивании. Малое отрицательное PI возможно из-за шума, а также потому что использование признака вредит этой модели на этой выборке. Это не «защитный эффект» признака. Удаление признаков требует переобучения и новой корректной оценки; все связанные признаки одновременно удалять по одному рейтингу опасно.

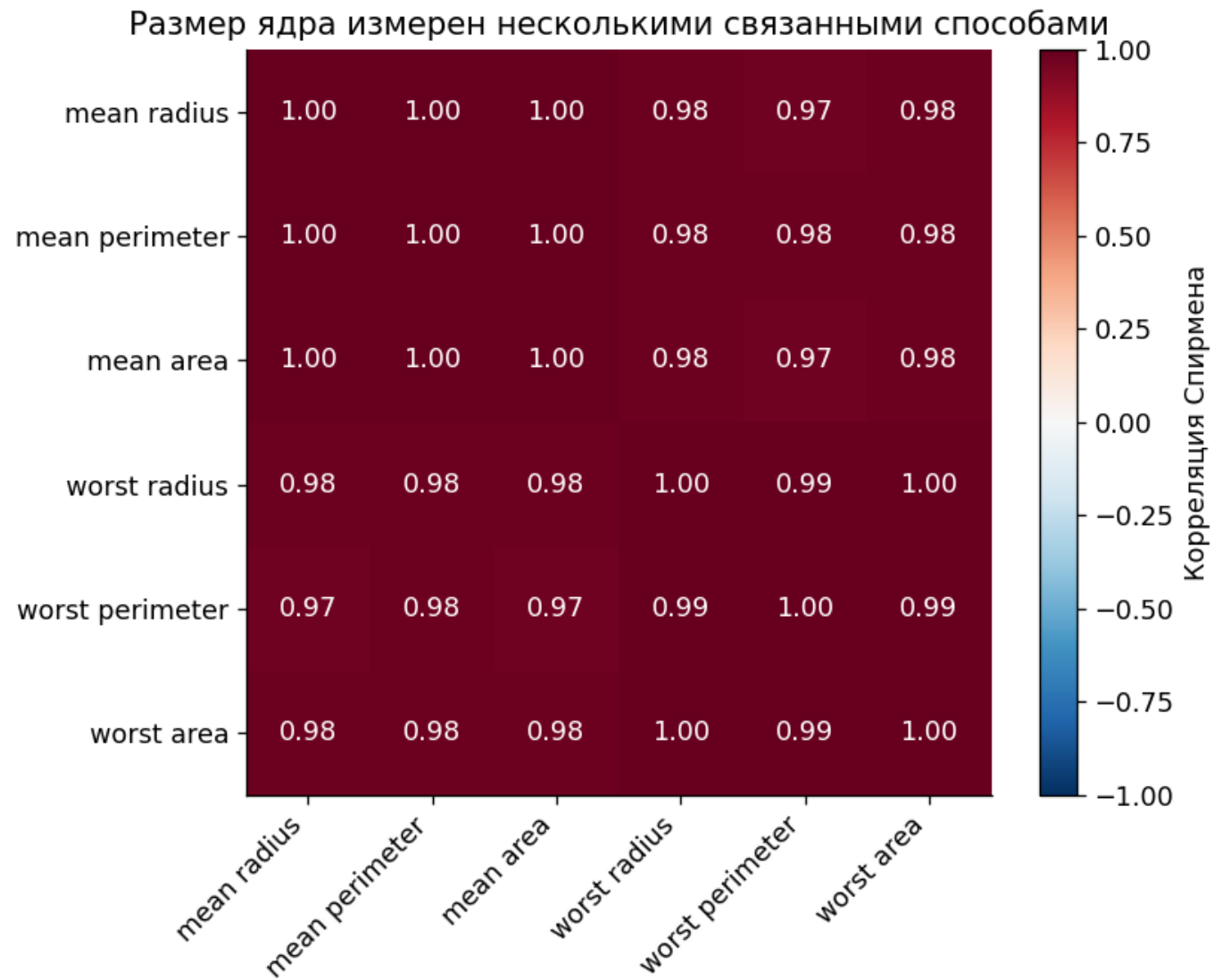
Связанные реальные признаки

Радиус, периметр и площадь описывают близкие свойства одного объекта. Посмотрим их связь **на train**. Низкая отдельная важность может означать малую потребность модели именно в этой колонке при наличии других. Точное поведение зависит от модели; дерево не обязано делить важность поровну.

Код 12

```
size_features = ["mean radius", "mean perimeter", "mean area", "worst radius", "worst perimeter", "worst area"]
corr = X_train[size_features].corr(method="spearman")
fig, ax = plt.subplots(figsize=(7.5, 6))
im = ax.imshow(corr, vmin=-1, vmax=1, cmap="RdBu_r")
ax.set_xticks(range(len(corr)), corr.columns, rotation=45, ha="right")
ax.set_yticks(range(len(corr)), corr.index)
for i in range(len(corr)):
    for j in range(len(corr)):
        ax.text(j, i, f"{corr.iloc[i,j]:.2f}", ha="center", va="center", color="white" if abs(corr.iloc[i,j])>.65 else "black")
ax.set_title("Размер ядра измерен несколькими связанными способами")
fig.colorbar(im, ax=ax, label="Корреляция Спирмена")
```

```
plt.tight_layout()
save_plot("05_correlations")
```



4. Одинаковые прогнозы — разные важности

Для контролируемой проверки возьмём **реальный mean radius и его точную копию**. Объекты и метки остаются реальными. Меняем только представление одного признака. При одинаково хороших сплитах дерево может выбрать любую из двух колонок.

До запуска: если первое дерево использовало оригинал, а второе — копию, какими будут MDI? Как поведёт себя PI для неиспользуемой колонки?

Код 13

```
X_duplicate = X[["mean radius"]].copy()
X_duplicate["mean radius copy"] = X_duplicate["mean radius"]
duplicate_models = {}
for seed in range(20):
    m = DecisionTreeClassifier(max_depth=1, random_state=seed).fit(X_duplicate.loc[train_idx], y_train)
    root_feature = int(m.tree_.feature[0])
    duplicate_models.setdefault(root_feature, (seed, m))
    if len(duplicate_models) == 2:
        break
assert set(duplicate_models) == {0, 1}, "Не удалось получить оба варианта сплита"
duplicate_rows = []
for feature_id, (seed, m) in sorted(duplicate_models.items()):
    p = permutation_importance(m, X_duplicate.loc[val_idx], y_val,
                              scoring="balanced_accuracy", n_repeats=20, random_state=SEED)
    for j, col in enumerate(X_duplicate.columns):
        duplicate_rows.append({"Дерево использует": X_duplicate.columns[feature_id], "Колонка": col,
                              "MDI": m.feature_importances_[j], "PI": p.importances_mean[j],
                              "validation": metrics(m, X_duplicate.loc[val_idx], y_val)["balanced_accuracy"]})
left, right = [v[1] for _, v in sorted(duplicate_models.items())]
print("Прогнозы совпадают для всех 569 объектов:", np.array_equal(left.predict(X_duplicate), right.predict(X_duplicate)))
display(pd.DataFrame(duplicate_rows).round(3))
```

Прогнозы совпадают для всех 569 объектов: True

	Дерево использует	Колонка	MDI	PI	validation
0	mean radius	mean radius	1.0	0.372	0.865
1	mean radius	mean radius copy	0.0	0.000	0.865
2	mean radius copy	mean radius	0.0	0.000	0.865
3	mean radius copy	mean radius copy	1.0	0.372	0.865

Задание 4. Сделайте вывод. Как получается, что в первом дереве MDI копии равен нулю, а во втором — единице? Означает ли нулевая PI, что колонка вообще не содержит сигнала?

Разбор для преподавателя: Обе колонки содержат один и тот же сигнал. MDI зависит от того, какую колонку выбрали при равных сплитах; выбранная получает 1, другая 0. PI конкретного дерева также падает при перемешивании используемой колонки и остаётся 0 для неиспользуемой. Нулевая важность для этой модели не равна отсутствию предсказательного сигнала в данных. PI тоже не устраняет проблему взаимозаменяемых признаков автоматически.

5. ELI5 и SHAP

ELI5: компактная таблица permutation importance

ELI5 — библиотека отображения и расчёта объяснений, а не отдельное определение важности. Ниже используем ту же идею перемешивания и ту же метрику, что выше. Реализации используют свои последовательности перемешиваний, поэтому числа могут немного различаться.

Код 14

```
import eli5
from eli5.sklearn import PermutationImportance
eli_pi = PermutationImportance(model, scoring="balanced_accuracy", n_iter=20,
                               random_state=SEED, cv="prefit")
eli_pi.fit(X_val[colums], y_val)
display(eli5.show_weights(eli_pi, feature_names=colums, top=8))
```

Weight	Feature
0.1552 ± 0.0572	worst perimeter
0.0884 ± 0.0481	worst concave points
0.0454 ± 0.0501	mean texture
0.0254 ± 0.0297	worst texture
0 ± 0.0000	worst symmetry
0 ± 0.0000	worst concavity
0 ± 0.0000	worst smoothness

Weight	Feature
0 ± 0.0000	worst compactness
... 22 more ...	

SHAP: почему такой прогноз для одного объекта?

Теперь объясняем **выход модели для класса 1**, а не качество всего классификатора:

$$f(x) = \mathbb{E}[f(X)] + \sum_j \phi_j(x).$$

Положительный вклад увеличивает выход модели относительно базового значения, отрицательный уменьшает. В этом примере выход — вероятность класса 1, поэтому вклад 0,10 означает 10 процентных пунктов выхода модели.

Для фона берём 100 объектов **из train**. Используем `interventional TreeSHAP`. Способ учёта зависимостей и выбор фона влияют на распределение вкладов, особенно при связанных признаках. Из SHAP не следует причинная интерпретация.

Код 15

```
import shap
background = X_train[columns].sample(n=min(100, len(X_train)), random_state=SEED)
explainer = shap.TreeExplainer(model, data=background,
                               feature_perturbation="interventional", model_output="probability")
explanation = explainer(X_val[columns])
# sklearn DecisionTreeClassifier возвращает объяснения для двух классов.
class_one = explanation[:, :, 1]
probabilities = model.predict_proba(X_val[columns])[:, 1]
residual = np.max(np.abs(class_one.base_values + class_one.values.sum(axis=1) - probabilities))
print(f"Проверка суммы вкладов: максимальная ошибка {residual:.2e}")
assert residual < 1e-5

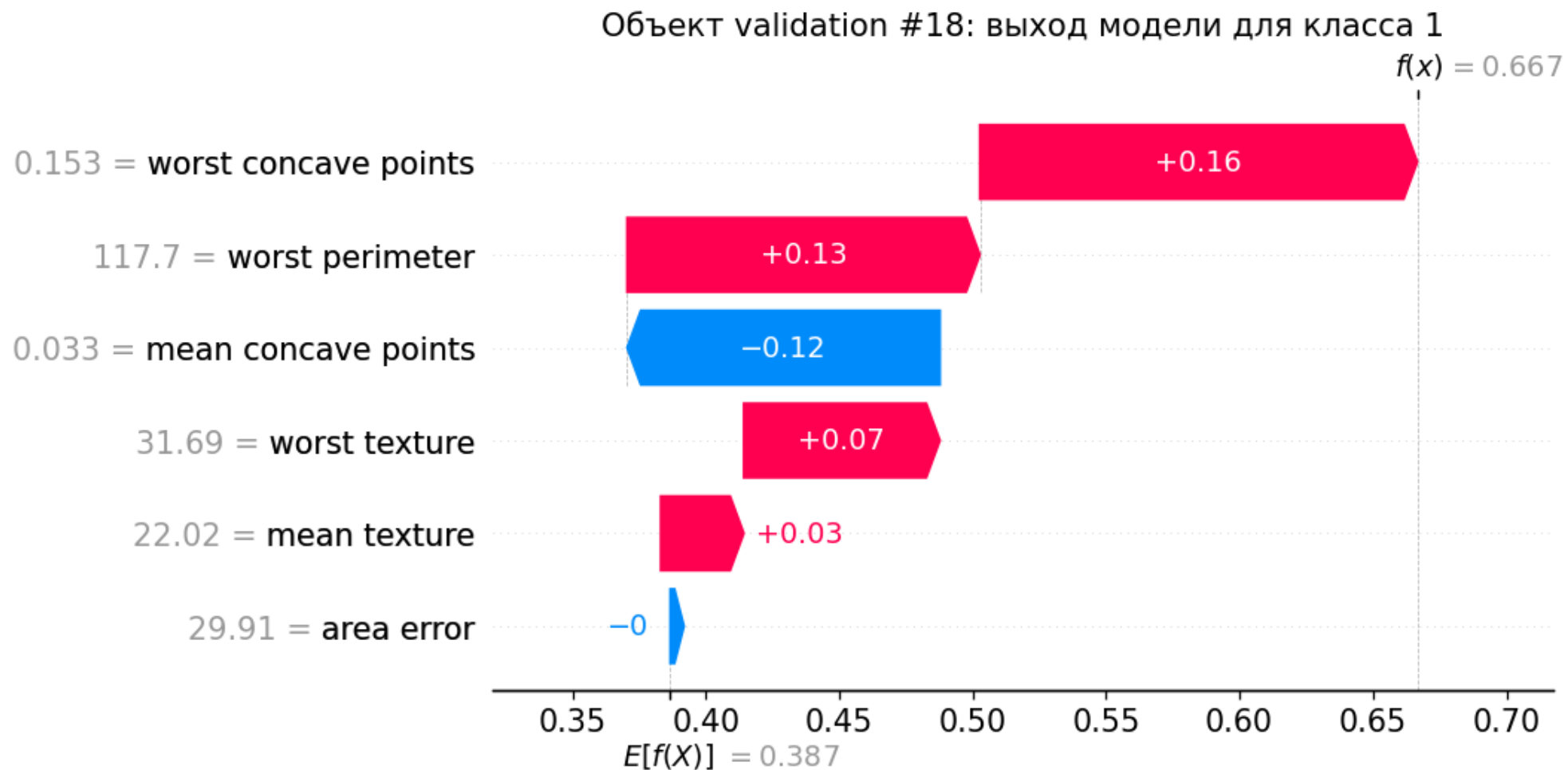
example_row = int(np.argmin(np.abs(probabilities - .5)))
active = np.flatnonzero(np.abs(class_one.values[example_row]) > 1e-8)
local_explanation = shap.Explanation(values=class_one.values[example_row, active],
                                   base_values=float(class_one.base_values[example_row]),
                                   data=X_val[columns].iloc[example_row, active].to_numpy(),
                                   feature_names=[columns[i] for i in active])
```

```

shap.plots.waterfall(local_explanation, max_display=8, show=False)
plt.title(f"Объект validation #{example_row}: выход модели для класса 1")
save_plot("06_shap_waterfall")
print("Базовый выход:", round(float(class_one.base_values[example_row]), 3))
print("Выход для объекта:", round(float(probabilities[example_row]), 3))

```

Проверка суммы вкладов: максимальная ошибка 3.01e-08
 Базовый выход: 0.387
 Выход для объекта: 0.667



Задание 5. Объясните один объект. Найдите самый большой положительный и отрицательный вклад на waterfall. Затем поменяйте `example_row` на другой объект и повторите график. Почему локальные вклады не обязаны совпадать с глобальным рейтингом PI?

Разбор для преподавателя: PI — изменение качества на наборе при нарушении колонки. SHAP распределяет отклонение выхода одного объекта от среднего по фону. Поэтому знак локального вклада и глобальный рейтинг не обязаны совпадать. У дерева вероятность определяется частотой класса в листе и не обязательно калибрована. Один большой вклад не доказывает причинность или переносимость вывода на другое население.

6. Конфиги и MLflow

Ручное изменение параметров удобно для первого эксперимента. Чтобы повторить результат, сохраняем **конфиг, данные/разбиение, версии библиотек, метрики и артефакты**.

MLflow хранит запуски в одном эксперименте. В этой практике сервер не нужен: записываем результаты локально и читаем таблицу запусков из ноутбука.

Задание 6. После трёх готовых запусков добавьте четвёртый конфиг. Меняйте только один параметр относительно исходного, сравните по validation и объясните, что именно изменилось. Test на этом шаге не используйте.

Код 16

```
import mlflow
import joblib
mlflow.set_tracking_uri(f"sqlite:///{{(OUT / 'mlflow.db')}.resolve()}")
experiment = mlflow.set_experiment("ITMO_Trees_WDBC")
versions = {p: importlib.metadata.version(p) for p in
            ["scikit-learn", "numpy", "pandas", "shap", "eli5", "mlflow"]}
versions["python"] = platform.python_version()
data_sha256 = hashlib.sha256(data.to_csv(index=False).encode()).hexdigest()
split_record = {"train": train_idx.tolist(), "validation": val_idx.tolist(), "test": test_idx.tolist()}

def run_config(config):
    cols = two_features if config["features"] == "two" else list(X.columns)
    m = DecisionTreeClassifier(max_depth=config["max_depth"], min_samples_leaf=config["min_samples_leaf"],
                              ccp_alpha=config["ccp_alpha"], random_state=config["model_seed"])
    m.fit(X_train[cols], y_train)
    scores = {f"{part}_{key}": value for part, xx, yy in
              [("train", X_train, y_train), ("validation", X_val, y_val)]}
```

```

        for key, value in metrics(m, xx[cols], yy).items()
with mlflow.start_run(run_name=config["run_name"]) as run:
    mlflow.log_params({k: "None" if v is None else v for k, v in config.items()})
    mlflow.set_tags({"data_sha256": data_sha256, "split": "60/20/20 stratified", "class_1": "malignant"})
    mlflow.log_metrics({"**scores", "n_leaves": m.get_n_leaves(), "actual_depth": m.get_depth()})
    mlflow.log_dict(config, "config.json")
    mlflow.log_dict(split_record, "split_indices.json")
    mlflow.log_dict(versions, "versions.json")
    mlflow.log_dict({"columns": cols, "target": {"0": "benign", "1": "malignant"}}, "schema.json")
    local_model = OUT / f"{run.info.run_id}.joblib"
    joblib.dump(m, local_model)
    mlflow.log_artifact(str(local_model), artifact_path="model")
    fig, ax = plt.subplots(figsize=(7, 3))
    ax.bar(["train", "validation"], [scores["train_balanced_accuracy"], scores["validation_balanced_accuracy"]],
           color=["#147d92", "#b94639"])
    ax.set(ylim=(0, 1.05), ylabel="Balanced accuracy", title=config["run_name"])
    mlflow.log_figure(fig, "quality.png")
    plt.close(fig)
return run.info.run_id

run_ids = []
for name, depth, leaf in [("stump", 1, 1), ("shallow", 3, 5), ("deep", None, 1)]:
    config_path = ROOT / "configs" / f"{name}.json"
    cfg = json.loads(config_path.read_text()) if config_path.exists() else {
        "run_name": name, "features": "all", "max_depth": depth, "min_samples_leaf": leaf,
        "ccp_alpha": 0.0, "split_seed": SEED, "model_seed": SEED}
    assert cfg["split_seed"] == SEED, "Для сравнения сохраняем одно разбиение"
    run_ids.append(run_config(cfg))

new_config = {"run_name": "my_leaf10", "features": "all", "max_depth": None,
              "min_samples_leaf": 10, "ccp_alpha": 0.0, "split_seed": SEED, "model_seed": SEED}
run_ids.append(run_config(new_config))
logged_runs = mlflow.search_runs(experiment_ids=[experiment.experiment_id])
logged_runs = logged_runs[logged_runs.run_id.isin(run_ids)]
display(logged_runs[["tags.mlflow.runName", "params.max_depth", "params.min_samples_leaf",
                    "metrics.train_balanced_accuracy", "metrics.validation_balanced_accuracy",
                    "metrics.n_leaves"]].sort_values("metrics.validation_balanced_accuracy", ascending=False).round(3))

```

	tags.mlflow.runName	params.max_depth	params.min_samples_leaf	metrics.train_balanced_accuracy	metrics.validation_balanced_accuracy	metrics.n_leaves
0	my_leaf10	None	10	0.962	0.939	8.0
3	stump	1	1	0.918	0.916	2.0

	tags.mlflow.runName	params.max_depth	params.min_samples_leaf	metrics.train_balanced_accuracy	metrics.validation_balanced_accuracy	metrics.n_leaves
2	shallow	3	5	0.965	0.914	7.0
1	deep	None	1	1.000	0.904	17.0

В архиве тот же эксперимент можно запустить из терминала:

```
python train.py --config configs/shallow.json
python -m mlflow ui --backend-store-uri sqlite:///runs/mlflow.db --host 127.0.0.1 --port 5000
```

В Jupyter на своём компьютере интерфейс открывается по адресу <http://127.0.0.1:5000>. В Colab используйте таблицу `search_runs` выше: она позволяет выполнить всё задание без настройки доступа к серверу.

Разбор для преподавателя: Конфиг фиксирует параметры, но сам по себе не фиксирует данные, разбиение и версии ПО. Логируем всё вместе. Сравнение возможно, когда меняется именно изучаемая настройка, а разбиение сохраняется. Самая высокая train-метрика не является критерием выбора. После нескольких запусках по одной validation возрастает риск подстроиться под неё; при серьёзном поиске нужна CV и независимая итоговая оценка.

Финальная модель и test

Теперь фиксируем выбор **до просмотра test**: сравниваем шесть исходных кандидатов и четыре запуска MLflow по validation; при равенстве предпочитаем меньше листьев. Дерево победившего запуска MLflow заново обучаем на исходном train с тем же конфигом. Ниже оцениваем **только этого победителя** на test и строим confusion matrix. Полученный результат — итог этой серии экспериментов, а не новый сигнал для подбора.

Код 17

```
final_options = [{"name": c["name"], "model": c["model"], "cols": c["cols"],
                  "validation": c["validation"], "leaves": c["leaves"]} for c in candidates]
for _, row in logged_runs.iterrows():
    cfg = mlflow.artifacts.load_dict(f"runs:{row.run_id}/config.json")
    cols = two_features if cfg["features"] == "two" else list(X.columns)
    m = DecisionTreeClassifier(max_depth=cfg["max_depth"], min_samples_leaf=cfg["min_samples_leaf"],
                              ccp_alpha=cfg["ccp_alpha"], random_state=cfg["model_seed"])
    m.fit(X_train[cols], y_train)
    final_options.append({"name": cfg["run_name"], "model": m, "cols": cols,
                        "validation": float(row["metrics.validation_balanced_accuracy"]),
                        "leaves": m.get_n_leaves()})
final = sorted(final_options, key=lambda c: (-c["validation"], c["leaves"]))[0]
```

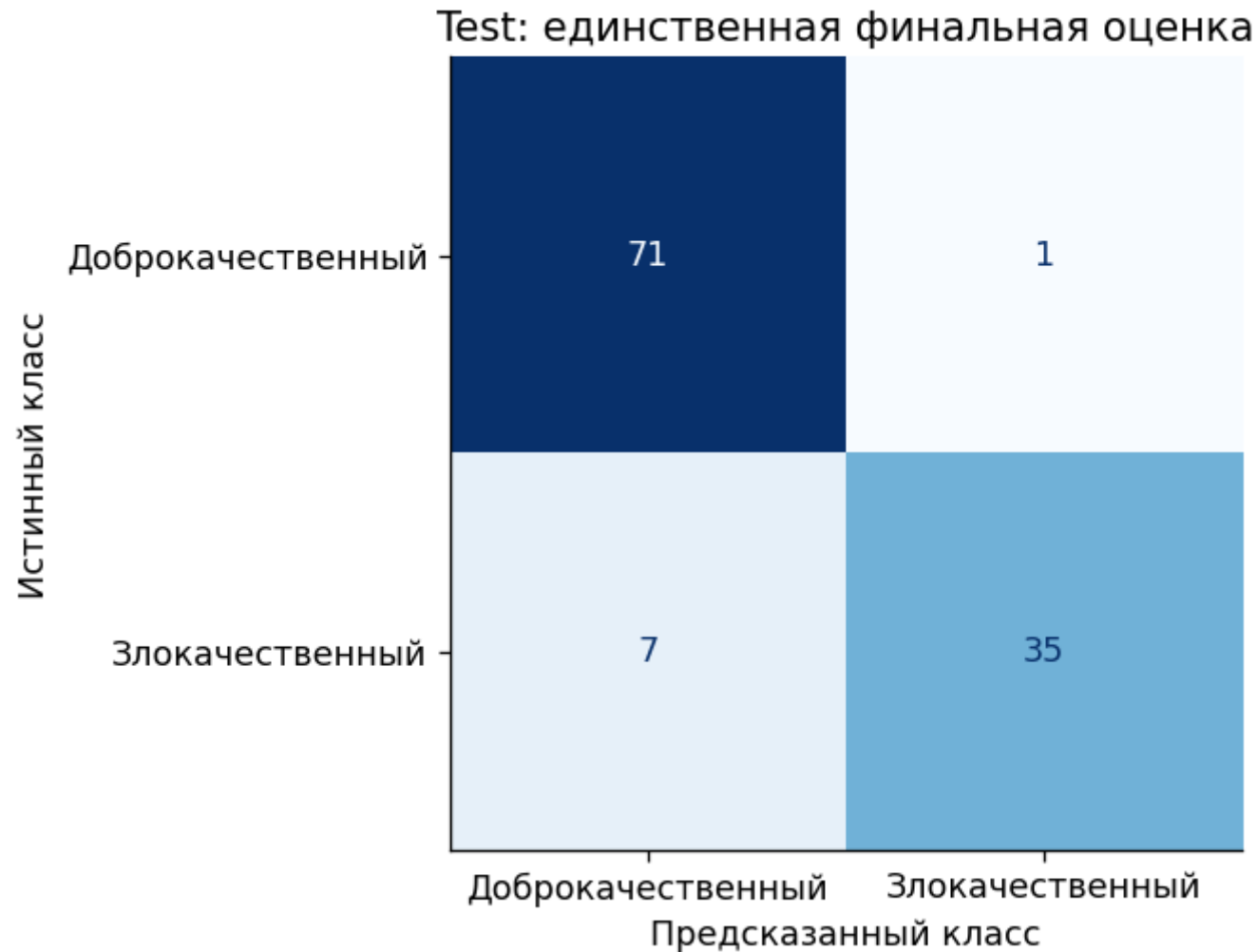
```

print("Фиксируем модель:", final["name"])
test_scores = metrics(final["model"], X_test[final["cols"]], y_test)
display(pd.DataFrame([test_scores], index=["test"]).round(3))
fig, ax = plt.subplots(figsize=(6, 4.7))
ConfusionMatrixDisplay.from_predictions(y_test, final["model"].predict(X_test[final["cols"]]),
                                       display_labels=CLASS_NAMES, cmap="Blues", colorbar=False, ax=ax)
ax.set_title("Test: единственная финальная оценка")
ax.set_xlabel("Предсказанный класс")
ax.set_ylabel("Истинный класс")
save_plot("07_test_confusion")
(OUT / "final_result.json").write_text(json.dumps({"name": final["name"], "metrics": test_scores,
                                                  "validation": final["validation"]}, indent=2))

```

Фиксируем модель: my_leaf10
 Out[17]: 203

	balanced_accuracy	accuracy	recall_class_1
test	0.91	0.93	0.833



Что должно остаться после практики

- Сложность дерева ограничивают глубина, размер листа и подрезка.
- Идеальный train может сочетаться с заметными ошибками на новых объектах.
- MDI, PI и SHAP отвечают на разные вопросы; взаимозаменяемые признаки затрудняют интерпретацию.
- Важность не доказывает причинность и не гарантирует отсутствие сигнала при нулевом значении.

- Настройки выбирают по validation; test используют после фиксации выбора.
- Воспроизводимый запуск сохраняет параметры, данные/разбиение, версии, метрики и артефакты.

Выходной вопрос: коллега предлагает оставить дерево с train accuracy = 1 и удалить все признаки с нулевым MDI. Какие две проверки вы попросите провести?

Разбор для преподавателя: 1. Проверить качество на отложенных данных и сравнить с более простыми деревьями при одном разбиении. 2. Проверить, почему признаки имеют нулевой MDI: модель могла выбрать связанные альтернативы. Если меняем набор признаков, переобучаем и оцениваем корректно, не используя итоговый test для нового подбора.

Источники

- [UCI WDBC](#): данные и описание признаков, CC BY 4.0.
- [scikit-learn](#): деревья.
- [scikit-learn](#): permutation importance.
- [scikit-learn](#): связанные признаки.
- [SHAP TreeExplainer](#).
- [ELI5 permutation importance](#).
- [MLflow Tracking](#).